

Patrick Baker

CAP 4773

DS Final Project

Data set: Bank Personal Loan

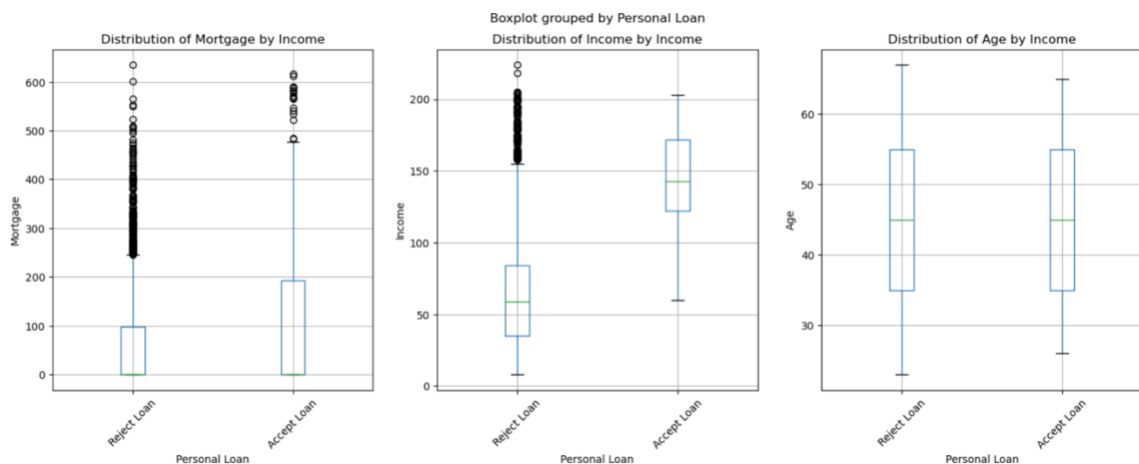
<https://www.kaggle.com/datasets/maryammanoochehry/bank-personal-loan>

Description: The dataset contains information about individuals and their interactions with a bank regarding personal loans. The "Personal Loan" column serves as the target variable, indicating whether an individual accepted a personal loan offer or not. Each row represents a unique individual. This dataset contains 14 predictors and features. Some of the predictors that are used within this dataset are income, age, ID, personal loan, credit cards, and experience and mortgage. In this dataset, there are many different observations. There are in total 500 observations and data points within this dataset. The variable types are integers and objects. This dataset contains predominately more integer variable types than object ones because most of the data is numeric. The problem that must be solved is for the bank to predict which customer would take a loan offered from it and which ones wouldn't. The approach that can be taken to address this problem is by developing strategic models that will indicate and predict which customer will and would not accept a loan offer. Several models that will be used for analysis are Multiple Linear Regression, Multiple Logistic Regression, Linear Regression using Regularization, Polynomial Regression, LDAs, Decision Tree for classification and for regression as well as Random Forest Classifier and a couple more.

	ID	Age	Experience	Income	ZIP Code	Family	CCAvg	Education	Mortgage	Personal Loan	Securities Account	CD Account	Online	CreditCard
0	1	25	1	49	91107	4	1/60	1	0	0	1	0	0	0
1	2	45	19	34	90089	3	1/50	1	0	0	1	0	0	0
2	3	39	15	11	94720	1	1/00	1	0	0	0	0	0	0
3	4	35	9	100	94112	1	2/70	2	0	0	0	0	0	0
4	5	35	8	45	91330	4	1/00	2	0	0	0	0	0	1

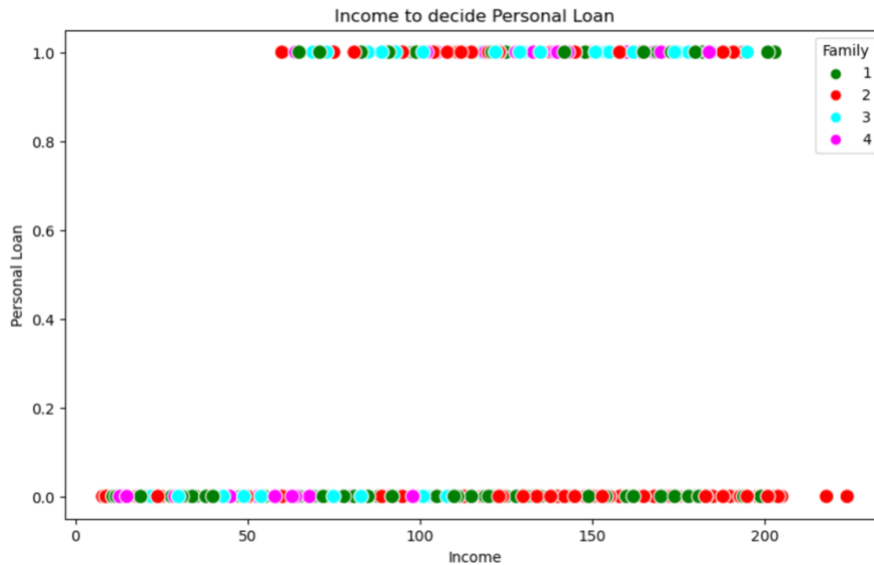
Creating features or encoding categorical values.

Prints first 5 rows of the dataset with all predictors shown.



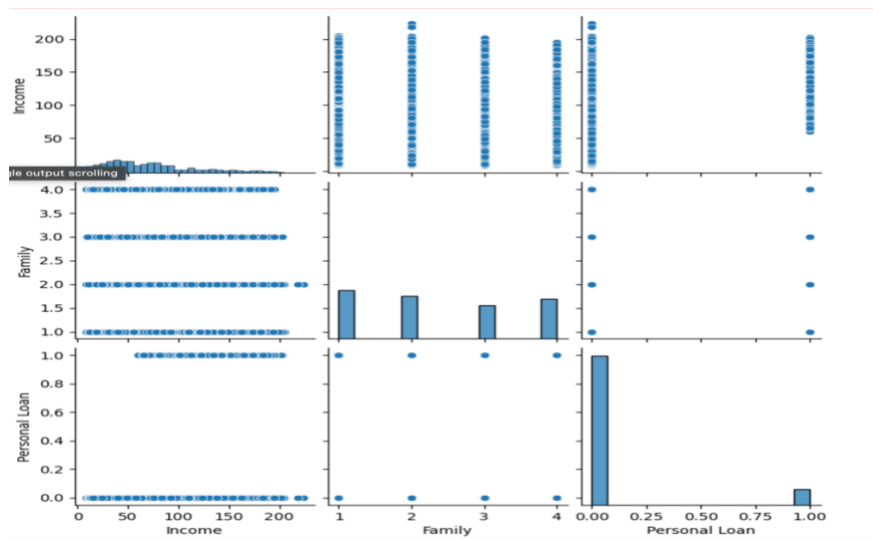
Box Plot

This Box plot shows the distribution of Mortgage, Income, and Age. Many customers with a higher mortgage will accept a loan than won't. In the distribution of Income, customers with an income between 120 and 175 will accept a loan and customers with an Income between 40 and 80 will reject a personal loan. For distribution of Age, customers that are between 35 and 55 years old will accept a loan and reject a loan.



Scatterplot

This scatterplot shows a family that consists of 1 person all the way up to a family with 4 people. A family that consists of 1 person is labeled green. A family that consists of 2 people are labeled red. A family that has three people are labeled blue and a family of four is pink. In this model many families either reject or accept a personal loan. In this model, there are more families that reject a loan than accept one with an income between 0 and 250. No families with an income between 0 and 50 accept a loan. There are many families that have 2 people that rejects a loan than accepts. Most families with 2 people make over 100 in income that does not need a loan. The families that have 4 people shows up the least within this model. The families that make the most money around 250 is red.



Scatterplot 2

```

Coefficients: [ 0.          0.14769463  0.04312705  0.07930468 -0.07113185 -0.00459492
 0.01455005]
Intercept: 0.09375
Mean Squared Error (MSE): 0.06472829500056188
R-squared (R2): 0.31121793029463285

```

Multi – linear regression

Coefficients: These values represent the weights assigned to each independent variable in the model.

For example, the coefficient for the second independent variable is 0.14776771, indicating that a one-

unit increase in this variable is associated with an increase of approximately 0.14776771 units in the dependent variable, holding other variables constant. Intercept: This is the value of the dependent variable when all independent variables are zero. In this case, the intercept is 0.09375.

Mean Squared Error (MSE): This measures the average squared difference between the actual values and the predicted values by the model. A lower MSE indicates a better fit of the model to the data.

R-squared (R2): This is a measure of how well the independent variables explain the variation in the dependent variable. It ranges from 0 to 1, with 1 indicating a perfect fit. In this case, the R-squared value is 0.3114, suggesting that approximately 31.14% of the variation in the dependent variable is explained by the independent variables in the model.

```
Lasso Regression Model
Coefficients: [ 0.          0.45198361  0.08829939  0.          0.          -0.
 0.01082357]
Intercept: -9.441878440092459e-17
Mean Squared Error (MSE): 0.06576174079355517
R-squared (R2): 0.30022090137211843
```

Linear Regression using Regularization.

This model is a Lasso Regression Model, which is a type of linear regression that includes a penalty term to enforce sparsity in the coefficient estimates. Here's what each component means:

Coefficients: Like the coefficients in a regular linear regression model, these values represent the weights assigned to each independent variable. However, in Lasso regression, the penalty term encourages some coefficients to be exactly zero, effectively performing variable selection. In your model, some coefficients are indeed zero, indicating that certain independent variables are not contributing to the prediction.

Intercept: This is the value of the dependent variable when all independent variables are zero, like the intercept in a regular linear regression model. Mean Squared Error (MSE): This measures the average squared difference between the actual values and the predicted values by the model. It's a measure of the model's accuracy, where lower values indicate a better fit to the data. R-squared (R^2): This is a measure of how well the independent variables explain the variation in the dependent variable. Like in the previous model, it ranges from 0 to 1, with 1 indicating a perfect fit. In this case, the R-squared value is 0.3002, indicating that approximately 30.02% of the variation in the dependent variable is explained by the independent variables in the model. The key difference between Lasso regression and regular linear regression lies in the penalty term, which encourages simpler models with fewer non-zero coefficients, effectively performing variable selection and potentially improving the model's interpretability.

Mean Squared Error for 3rd-degree Polynomial: 0.05
Mean Squared Error for 4th-degree Polynomial: 0.05
Mean Squared Error for 5th-degree Polynomial: 0.07

Polynomial Regression

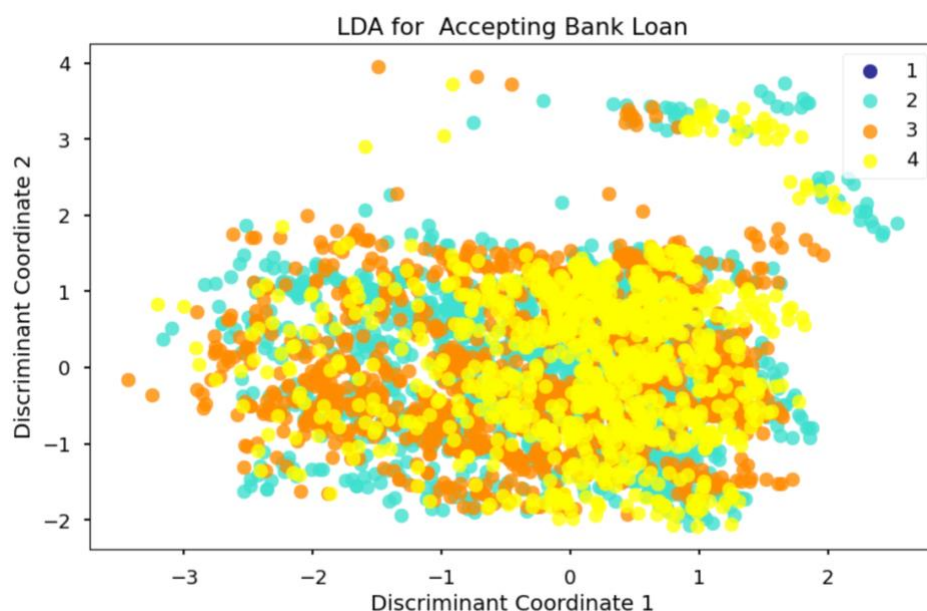
Mean Squared Error (MSE): This is a measure of the average squared difference between the actual values and the predicted values by the model. It indicates how well the model fits the data, with lower values indicating better fit.

3rd-degree Polynomial: A 3rd-degree polynomial regression model is a type of regression where the relationship between the independent variable(s) and the dependent variable is modeled as a polynomial function of degree 3. The MSE for this model is 0.05.

4th-degree Polynomial: Similarly, a 4th-degree polynomial regression model is a type of regression where the relationship between the independent variable(s) and the dependent variable is modeled as a polynomial function of degree 4. The MSE for this model is also 0.05.

5th-degree Polynomial: This refers to a polynomial regression model of degree 5. The MSE for this model is 0.07.

Comparing the MSE values, we can see that the 3rd-degree and 4th-degree polynomial models have the same MSE, while the 5th-degree polynomial model has a slightly higher MSE. This suggests that the 3rd and 4th-degree polynomial models might be preferable over the 5th-degree polynomial model, as they provide similar accuracy with potentially simpler models. However, other factors such as model complexity and interpretability should also be considered when selecting the best model.



Linear Discriminant Analysis.

This LDA Model shows families that consists of 1 person up to 4 people. Families of one is blue. Families of 2 are light blue. Families of 3 are orange and families of 4 are yellow. In this model we see more families rejecting a loan from the bank then accepting it because most of the families that reject a loan start at 0 and go back. LDA is particularly useful when the classes are well-separated, and the assumptions of normality and equal covariance matrices hold. However, if these assumptions are

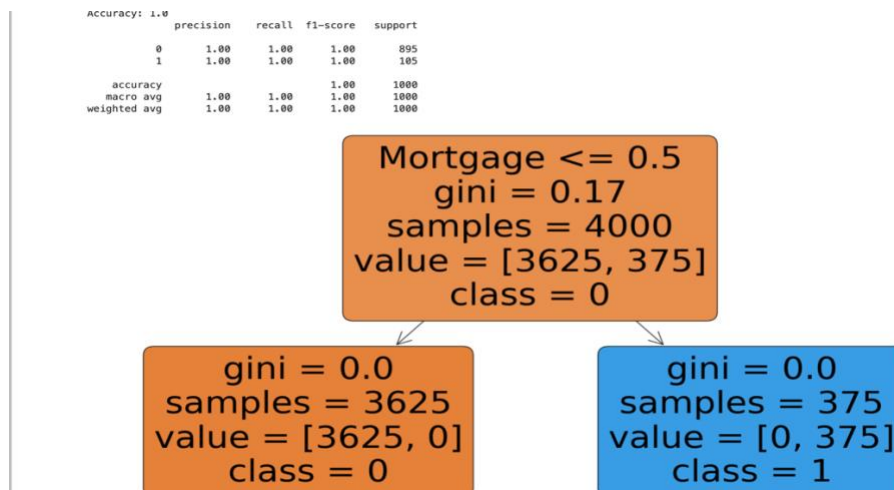
violated, other methods like logistic regression or decision trees might be more appropriate.

Additionally, it's important to ensure that the features used are relevant and not biased.

	Variable	VIF
0	const	440.901277
1	Age	93.265128
2	Experience	93.163041
3	Education	1.077631
4	Family	1.008624

341: #Analysis

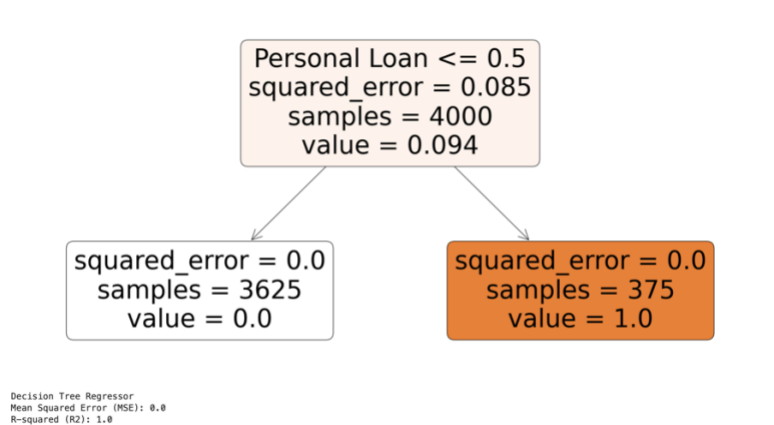
VIF is a measure of the amount of multicollinearity in regression analysis. Multicollinearity exists when there is a correlation between multiple independent variables in a multiple regression model. This can adversely affect the regression results. const: This likely represents a constant or intercept term in the regression model. A high VIF value for the constant suggests that there might be an issue with multicollinearity in the model. Age and Experience: Both these variables have high VIF values (above 10), indicating they are highly correlated with other variables in the model, possibly with each other. This high correlation might pose a problem for the regression analysis. Education and Family: These variables have low VIF values (close to 1), suggesting they have low multicollinearity with other variables in the model. This means they are less likely to cause issues in the regression analysis due to multicollinearity. In summary, based on the VIF values: There might be an issue with multicollinearity between the Age and Experience variables. Education and Family variables seem to be less affected by multicollinearity.



Decision Tree for classification

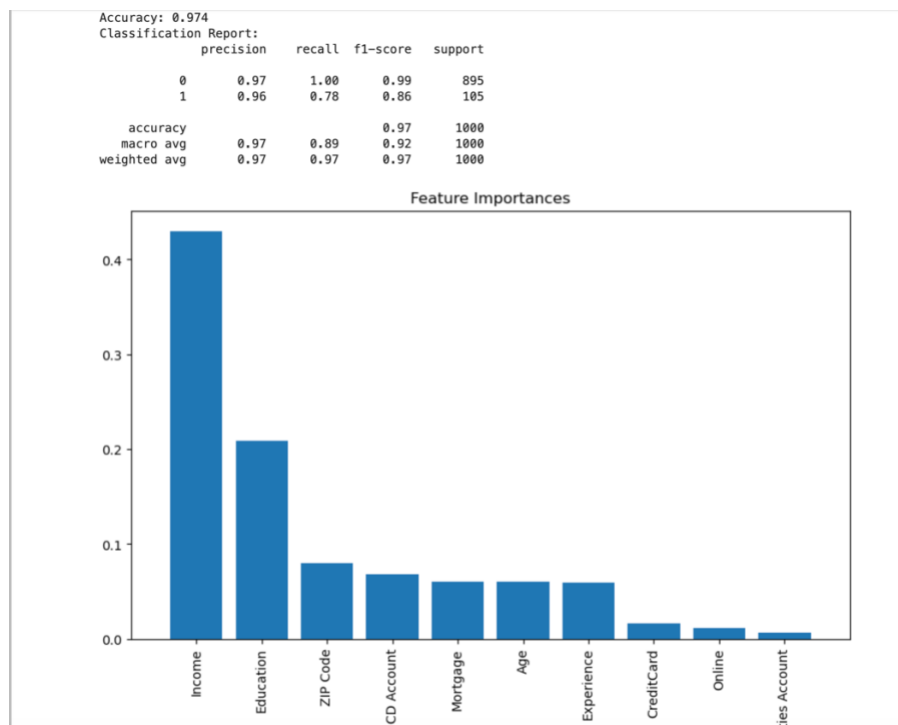
This decision tree has 2 levels. Precision for both 0 and 1 is 1.00. recall is 1.00 for both 0 and 1 which are customers rejecting and accepting loans. F 1 score is 1 for both 0 and 1 and support is 895 for 0 and 105 for 1. Accuracy is 1 for f1 score and 1000 for support. Macro avg. is 1 for precision and 1 for recall and 1 for f1 score and 1000 for support. Weighted avg is the same as macro avg. mortgage is less than and equal to 0.5. Gini is 0.17, samples are 4000 and value is 3625 and 375. Class is 0. This is for the top level. The bottom level, Gini is 0 for the leftmost and underneath it samples are 3625 and value is 3625 and 0. Class is also 0. For The rightmost, Gini is 0, samples are 375, value is 0 and 375 and class is 1.

Financial institutions use decision trees to predict the probability of loan acceptance by analyzing applicant data such as credit score, income, employment history, and debt-to-income ratio and mortgage. This helps in assessing credit risk and making informed lending decisions.



Decision tree for Regression

The very top node has personal loan less than 0.5 and squared error is 0.085 and samples are 4000 with value at 0.094. leftmost, squared error is 0 samples are 3625 and value is 0. Rightmost, squared error is 0. Samples are 375 and value is 1. Prediction: Once the tree is constructed, to make predictions for new data points, the algorithm traverses the tree from the root node down to a leaf node based on the values of the input features. The predicted value for the new data point is then determined by the average (or another measure of central tendency) of the response variable in the leaf node reached.



Random Forest classifier.

Precision is 0.97 for 0 and 0.96 for 1. Recall is 1 for 0 and 0.78 for 1. F1 score is 0.99 for 0 and 0.86 for 1.

Support is 895 for 0 and 105 for 1. Accuracy is 0.97 for f1 score and 1000 for support. Macro avg. is 0.97 for precision and 0.89 for recall and 0.92 for f1 score and 1000 for support. Weighted avg is 0.97 for both precision and recall and f1 score and 1000 for support. In the chart income goes up to 0.4. education is 0.2, zip code is 0.1, Cd account is just below 0.1, Mortgage is just below 0.1, as well as age, experience credit card online and securities account predictors. Overall, Random Forest classifier is a powerful and widely used machine learning algorithm for classification tasks across various domains, offering flexibility, robustness, and high predictive performance.

Decision Tree	
Index	Task 1
0	4
1	2
2	1
3	3
4	5

Make predictions and classifications with new data.

```

Cluster Profiling: (Predictor mean by clusters)
      Age  Experience  Income  ZIP Code  Family  Education \
Cluster
0      -0.853013    -0.847973    0.048629    0.025567    0.055051    -0.041189
1       0.870400     0.863673   -0.130441   -0.033879   -0.064992     0.037163
2       0.021777     0.033916    0.624442    0.059664    0.066872     0.038718

      Mortgage  Securities Account  CD Account  Online  CreditCard
Cluster
0      -0.014657      -0.066468   -0.252646   -0.063057   -0.067060
1      -0.033733      -0.096457   -0.252646   -0.026805   -0.068698
2       0.377309       1.273498    3.958114    0.707240    1.063291

Feature Importance: (Just absolute values)
      Age  Experience  Income  ZIP Code  Family  Education  Mortgage \
0  0.853542    0.848510    0.047328    0.025249    0.055255    0.040659    0.014374
1  0.870014    0.863301    0.129014    0.033523    0.065137    0.036579    0.034012
2  0.021777     0.033916    0.624442    0.059664    0.066872    0.038718    0.377309

      Securities Account  CD Account  Online  CreditCard
0      0.066321    0.252646    0.062446    0.066756
1      0.096591    0.252646    0.027449    0.069009
2      1.273498    3.958114    0.707240    1.063291

Cluster Validation Metrics:
Silhouette Score: 0.15893011735792276
Davies-Bouldin Score: 1.905361404076121

Anomaly Detection: (Distance threshold to the centroid for 95% of the data points)
Indices of Anomalies in the Original Dataset: [ 8 26 52 62 71 91 113 115 119 132 136 146 160 165
184 193 215 233 236 268 314 319 344 416 426 486 511 517
523 536 615 617 662 676 677 682 693 709 713 719 722 769
811 814 817 823 849 881 883 902 1002 1007 1017 1025 1046 1073
1117 1118 1142 1146 1155 1200 1201 1220 1237 1263 1276 1281 1292 1306
1316 1376 1459 1489 1494 1517 1535 1536 1582 1596 1616 1629 1661 1672
1711 1727 1729 1734 1743 1764 1767 1806 1807 1821 1870 1874 1880 1897
1905 1914 1919 1931 1934 1968 1971 2022 2023 2040 2044 2085 2090 2099
2152 2180 2194 2203 2240 2247 2260 2275 2311 2317 2319 2354 2357 2407
2415 2427 2431 2444 2477 2495 2496 2511 2562 2577 2607 2634 2645 2670
2673 2674 2731 2733 2743 2788 2822 2849 2870 2874 2876 2880 2887 2896
2929 2943 2954 2969 2987 2988 3035 3064 3070 3071 3073 3105 3128 3170
3182 3273 3299 3333 3341 3351 3356 3369 3380 3399 3403 3423 3424 3509
3519 3539 3578 3694 3730 3767 3812 3872 3897 3900 3914 3917 3921 3956
3965 3983 3984 3997]

```

K means Unsupervised Learning

The k-means algorithm is a popular unsupervised learning technique used for clustering data into distinct groups or clusters based on similarities in the feature space. Here's a breakdown of what k-means unsupervised learning model describes:

Clustering: The primary objective of k-means is to partition a dataset into k clusters, where each data point belongs to the cluster with the nearest mean (centroid). It seeks to minimize the intra-cluster variance, meaning that data points within the same cluster are as similar as possible to each other, while being as dissimilar as possible from data points in other clusters.

Centroid-based: K-means is a centroid-based algorithm, where each cluster is represented by the mean (centroid) of the data points belonging to that cluster. Initially, the centroids are randomly chosen from the data points, and then the algorithm iteratively updates the centroids until convergence.

Distance Metric: The distance metric used in k-means is typically Euclidean distance, although other distance metrics such as Manhattan distance or cosine similarity can also be used, depending on the nature of the data.

Initialization and Convergence: The k-means algorithm involves two main steps: (a) assignment of data points to the nearest centroid (cluster assignment step), and (b) updating the centroids based on the mean of the data points assigned to each cluster (centroid update step). These steps are repeated iteratively until convergence, where the centroids no longer change significantly between iterations, or a specified number of iterations is reached.

Number of Clusters (k): One of the key parameters in k-means is the number of clusters (k) to be formed. However, determining the optimal value of k can be challenging and often requires domain knowledge or the use of validation metrics such as the elbow method or silhouette score.

Applications: K-means is widely used in various applications, including customer segmentation, image compression, anomaly detection, document clustering, and pattern recognition. It's an efficient and scalable algorithm that can handle large datasets with many features.

In summary, k-means unsupervised learning model describes a clustering technique that partitions data into k clusters based on similarities in the feature space, with the goal of minimizing intra-cluster variance and maximizing inter-cluster variance.

Code

```
# 2.1.1 Load required libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm
from statsmodels.discrete.discrete_model import Logit

from sklearn import datasets
from sklearn.linear_model import LinearRegression, Lasso, LassoCV
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import cross_val_score, train_test_split, KFold
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import silhouette_score, davies_bouldin_score, mean_squared_error, r2_score
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from scipy.spatial.distance import cdist
from sklearn.pipeline import Pipeline

#2.1.2 Load the dataset
df = pd.read_csv('Bank_Personal_Loan.csv')
# Drop the 'CCAvg' column
df.drop(columns=['ID', 'CCAvg'], inplace=True)
```

Box Plots

```
# Box plot for three columns in the DataFrame
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(18, 6))

# List of predictors to plot
predictors = ['Mortgage', 'Income', 'Age']
target_names = ['Reject Loan', 'Accept Loan']

# Boxplot iteration over the predictors
for ax, predictor in zip(axes, predictors):
    df.boxplot(column=[predictor], by='Personal Loan', ax=ax, grid=True)
    ax.set_title(f'Distribution of {predictor} by Income')
    ax.set_xlabel('Personal Loan')
    ax.set_ylabel(f'{predictor}')
    ax.set_xticklabels(target_names, rotation=45) # Rotate class names for better visibility
```

#3.1.2. Scatter plot between two predictors

```
plt.figure(figsize=(10, 6))
palette = {
    0: 'blue',
    1: 'green',
    2: 'red',
    3: 'cyan',
    4: 'magenta',
    5: 'yellow',
    6: 'black'
}

sns.scatterplot(x='Income', y='Personal Loan', hue='Family', data=df, palette=palette, s=100)
plt.title('Income to decide Personal Loan')
plt.xlabel('Income')
plt.ylabel('Personal Loan')
plt.legend(title='Family')
plt.show()
```

```
] : # scatter plot
sns.pairplot(df[['Income', 'Family', 'Personal Loan']], diag_kind='hist')
plt.show()
```

```
# Select columns for X
X = df[['Age', 'Experience', 'Education', 'Family']]

# Add a constant column to the dataframe (required for VIF calculation)
X = add_constant(X)

# Calculate VIF values for each variable
vif_data = pd.DataFrame()
vif_data["Variable"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]

print(vif_data)
```

```
# Select predictor variables (x) and target variable (y)
X = df[['Income', 'Family', 'Age', 'Experience', 'Mortgage']]
X = sm.add_constant(X) # Adds a constant column to input features to account for the intercept
y = df['Personal Loan'] # Use 'medv' as the target variable

# Split the dataset into training and testing sets for validation
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

```
[200]: # FOR THE REGRESSION TASK
#5.1. Multiple Linear Regression

#5.1.1. Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)

#5.1.2. Train a linear regression model
linear_reg = LinearRegression()
linear_reg.fit(X_scaled, y_train)

#5.1.3. Standardizing the test set
X_test_scaled = scaler.transform(X_test)

#5.1.4. Predicting median_house_value using the trained model
y_pred = linear_reg.predict(X_test_scaled)

#5.1.7. Evaluate the model's performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

#5.1.8. Display the model statistics
print(f"Coefficients: {linear_reg.coef_}")
print(f"Intercept: {linear_reg.intercept_}")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R-squared (R2): {r2}")
```

```
]: #5.2. Linear Regression using Regularization

#5.2.1. Standardizing the features
scaler_X = StandardScaler()
X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

#5.2.2. Standardizing the target variable
scaler_y = StandardScaler()
y_train_scaled = scaler_y.fit_transform(y_train.values.reshape(-1, 1)).flatten() # Reshape for a single feature

#5.2.3. Training a Lasso regression model using some alpha
lasso_reg = Lasso(alpha=0.05)
lasso_reg.fit(X_train_scaled, y_train_scaled)

#5.2.4. Predicting 'mdcv' levels using the trained Lasso regression model
y_pred_scaled = lasso_reg.predict(X_test_scaled)

#5.2.5. Reversing the scaling of predictions to original scale
y_pred_lasso = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1)).flatten()

#5.2.6. Evaluating the model's performance
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

#5.2.7. Displaying the model statistics
print("Lasso Regression Model")
print(f"Coefficients: {lasso_reg.coef_}")
print(f"Intercept: {lasso_reg.intercept_}")
print(f"Mean Squared Error (MSE): {mse_lasso}")
print(f"R-squared (R2): {r2_lasso}")
```

#5.3. Polynomial Regression

5.3.1. Using a pipeline for 3rd-degree polynomial regression

```
poly3_pipeline = Pipeline([
    ('scaler', StandardScaler()), # First, scale the features
    ('poly', PolynomialFeatures(degree=3)), # Then, generate polynomial features
    ('linear', LinearRegression()) # Finally, apply linear regression
])
```

5.3.2. Fit the pipeline on the training data

```
poly3_pipeline.fit(X_train, y_train)
```

5.3.3. Predict on the test data

```
y_pred_poly3 = poly3_pipeline.predict(X_test)
```

5.3.4. Calculate MSE

```
mse_poly3 = mean_squared_error(y_test, y_pred_poly3)
```

5.3.5. Print the MSE for the 3rd-degree polynomial model

```
print(f"Mean Squared Error for 3rd-degree Polynomial: {mse_poly3:.2f}")
```

5.3.6. Using a pipeline for 4th-degree polynomial regression

```
poly4_pipeline = Pipeline([
    ('scaler', StandardScaler()), # First, scale the features
    ('poly', PolynomialFeatures(degree=4)), # Then, generate polynomial features
    ('linear', LinearRegression()) # Finally, apply linear regression
])
```

5.3.7. Fit the pipeline on the training data

```
poly4_pipeline.fit(X_train, y_train)
```

5.3.8. Predict on the test data

```
y_pred_poly4 = poly4_pipeline.predict(X_test)
```

5.3.9. Calculate MSE

```
mse_poly4 = mean_squared_error(y_test, y_pred_poly4)
```

5.3.10. Print the MSE for the 4th-degree polynomial model

```
print(f"Mean Squared Error for 4th-degree Polynomial: {mse_poly4:.2f}")
```

5.3.11. Using a pipeline for 5th-degree polynomial regression

```
poly5_pipeline = Pipeline([
```

```

# Data setup
d = df.dropna(subset=['CreditCard']) # Drop NAN from default column
X = df[['Age', 'Experience', 'Income', 'Family', 'Mortgage']]
#X = data.loc[:, data.columns != "CreditCard"] # Select all columns except the one you will
X = sm.add_constant(X)
y = df['Family']

#3.2. Fit the LDA model
lda = LinearDiscriminantAnalysis()
lda.fit(X, y)

#3.3. LDA Variance ratio
print(lda.explained_variance_ratio_)

#3.4. Transform the data using the fitted LDA in the new space
X_r_lda = lda.transform(X)

target_names = ['1', '2', '3', '4'] # Your classes name

#3.5. Set the style and create the figure
with plt.style.context('seaborn-talk'):
    fig, ax = plt.subplots(figsize=[10,6])
    colors = ['navy', 'turquoise', 'darkorange', 'yellow']
    for color, i, target_name in zip(colors, [0, 1, 2, 3], target_names):
        ax.scatter(X_r_lda[y == i, 0], X_r_lda[y == i, 1], alpha=.8, label=target_name, color=color)
    ax.set_title('LDA for Bank data')
    ax.set_xlabel('Discriminant Coordinate 1')
    ax.set_ylabel('Discriminant Coordinate 2')
    ax.legend(loc='best')
    plt.show()

[0.86369664 0.11548206 0.0208213 ]

```

LDA for Bank data

```

: # Creating features or encoding categorical values.
df['Family'] = df['Family'].replace({'1': 0, '2': 1, '3': 2, '4': 3})
df['Education'] = df['Education'].replace({'1': 0, '2': 1, '3': 2})
df['Personal Loan'] = df['Personal Loan'].replace({'0':0, '1':1})
df['Securities Account'] = df['Securities Account'].replace({'0':0, '1':1})
df['CD Account'] = df['CD Account'].replace({'0':0, '1':1})
df['Online'] = df['Online'].replace({'0':0, '1':1})
df['CreditCard'] = df['CreditCard'].replace({'0':0, '1':1})
df.head()

```

```

|: #5.6. Decision Tree for classification (TASK 1) ONLY THREE LEVELS

#5.6.1. Selecting a subset of the features
X = df.drop('Income', axis=1)
y = df['Personal Loan']

#5.6.2. Split the dataset in training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#5.6.3. Creating a Decision Tree Classifier
dtc = DecisionTreeClassifier(max_depth=3, random_state=42)

#5.6.4. Training the model
dtc.fit(X_train, y_train)

#5.6.5. Making predictions on the test data
y_pred = dtc.predict(X_test)

#5.6.6. Calculate the accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

#5.6.9. Print detailed classification metrics
print(classification_report(y_test, y_pred))

#5.6.10 Plotting the Decision Tree
feature_names = ["Age", "Income", "Family", "Experience", "IDEducation", "ZIP Code", "Mortgage", "Personal Loan"]
class_names = ['0', '1']
plt.figure(figsize=(20,10))
plot_tree(dtc, filled=True, feature_names=feature_names, class_names = class_names, rounded=True)
plt.show()

```

```

from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Creating a Decision Tree Regressor
regressor = DecisionTreeRegressor(max_depth=3, random_state=42)

# Training the model
regressor.fit(X_train, y_train)

# Convert feature names Index to list
feature_names = list(X.columns)

# Plotting the Decision Tree
plt.figure(figsize=(20,10))
plot_tree(regressor, filled=True, feature_names=feature_names, rounded=True)
plt.tight_layout() # Adjust layout to prevent overlapping
plt.show()

# Predicting the target variable in the test set
y_pred = regressor.predict(X_test)

# Mean Squared Error
mse = mean_squared_error(y_test, y_pred)

# R-squared Value
r2 = r2_score(y_test, y_pred)

# Displaying the statistics
print(f"Decision Tree Regressor")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R-squared (R2): {r2}")

```

```

8]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# Select features and target variable
X = df.drop(columns=['Family', 'Personal Loan']) # Excluding 'Family' and 'Personal Loan' for t
y = df['Personal Loan']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Creating a Random Forest Classifier
clf = RandomForestClassifier(n_estimators=100, random_state=42)

# Training the model
clf.fit(X_train, y_train)

# Making predictions
y_pred = clf.predict(X_test)

# Calculate the accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Print detailed classification metrics
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Feature importances
feature_importances = clf.feature_importances_
features = X.columns.tolist()
indices = np.argsort(feature_importances)[::-1]

# Plotting Feature Importances
plt.figure(figsize=(10,6))

```

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from scipy.spatial.distance import cdist
import pandas as pd
import numpy as np

# Function for cluster profiling
def cluster_profiling(X, labels, feature_names):
    df = pd.DataFrame(X, columns=feature_names)
    df['Cluster'] = labels
    profile = df.groupby('Cluster').mean()
    return profile

# Function for calculating feature importance
def feature_importance(kmeans, feature_names):
    centroids = kmeans.cluster_centers_
    importance = pd.DataFrame(centroids, columns=feature_names).abs()
    return importance

# Function for cluster validation using silhouette score
def cluster_validation(X, labels):
    return silhouette_score(X, labels)

# Anomaly detection function
def anomaly_detection(X, kmeans):
    distances = cdist(X, kmeans.cluster_centers_, 'euclidean')
    distance_to_nearest_centroid = np.min(distances, axis=1)
    outlier_threshold = np.percentile(distance_to_nearest_centroid, 95)
    anomaly_indices = np.where(distance_to_nearest_centroid > outlier_threshold)[0]
    return anomaly_indices

# Load the dataset
df = pd.read_csv('Bank_Personal_Loan.csv')

# Drop unnecessary columns
X = df.drop(columns=['ID', 'Personal Loan', 'CCAvg'])
y = df['Personal Loan']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale the data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Apply KMeans clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_train_scaled)
labels = kmeans.labels_

# Cluster Profiling
feature_names = X.columns.tolist()
profile = cluster_profiling(X_train_scaled, labels, feature_names)
print("Cluster Profiling: (Predictor mean by clusters)\n", profile)

# Feature Importance
importance = feature_importance(kmeans, feature_names)
print("\nFeature Importance: (Just absolute values)\n", importance)

# Evaluate cluster quality
silhouette_avg = cluster_validation(X_train_scaled, labels)
davies_bouldin = davies_bouldin_score(X_train_scaled, labels)
print("\nCluster Validation Metrics:")
print("Silhouette Score:", silhouette_avg)
print("Davies-Bouldin Score:", davies_bouldin)

# Anomaly Detection
anomaly_indices = anomaly_detection(X_train_scaled, kmeans)
print("\nAnomaly Detection: (Distance threshold to the centroid for 95% of the data points)")
print("Indices of Anomalies in the Original Dataset:", anomaly_indices)

# Filter the original DataFrame to get the data of the detected anomalies
anomalies = pd.DataFrame(X_train, columns=feature_names).iloc[anomaly_indices]
print("\nData of Detected Anomalies:\n", anomalies)

```

```

[43]: # Select predictor variables (x) and target variable (y)
X = df[['Income', 'Family', 'Age', 'Experience', 'Mortgage']]
X = sm.add_constant(X) # Adds a constant column to input features to account for the intercept
y = df['Personal Loan'] # Use 'medv' as the target variable

# Split the dataset into training and testing sets for validation
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```

#Step 7. DEPLOYMENT PHASE
#7.1. Make predictions and classifications with new data
# Decision Tree Classifier - TASK 1
# Logistic Regression - TASK 2
# Lasso Regression Model - TASK 3

#7.1.1. Manually building df_new based on the sampled data
X_new = random_points

# 7.1.2. Decision Tree Classifier - TASK 1
y_pred_dt = dtc.predict(X_new)
#print("TASK 1 - Decision Tree Predictions:\n", y_pred_dt)

# 7.1.5. Printing results
#Header
print(f"{'':<6} {'Decision Tree':<15}")
print(f"{'Index':<6} {'Task 1':<15}")

# Rows of the table
for index, dt in enumerate(y_pred_dt):
    print(f"{'index':<6} {'dt':<15}")

```